

MLOps Practical Troubleshooting Guide

Golden Rule: Always troubleshoot in order. Do not uninstall tools, reset Kubernetes, delete all images, or run random cleanup commands unless the guide specifically asks you to.

How to Use This Guide

1. Start with the Pre-Practical Health Check before beginning the practical.
2. When something fails, identify which layer is failing: Docker → Kubernetes → Helm → Prometheus → Grafana.
3. Run the diagnosis command shown in the relevant section.
4. Read the STATUS / error message before applying a fix.
5. After every fix, repeat the original command and verify that the problem is resolved.

1. Pre-Practical Health Check

Complete these checks before starting the practical. If one check fails, fix it before moving to the next layer.

1.1 Check Docker

Step 1 — Open Docker Desktop. Wait until Docker Desktop shows that the Docker engine is running.

Step 2 — Verify the Docker CLI.

```
docker version
```

Check: Both Client and Server information should appear.

Action: If only Client appears or you see 'Cannot connect to the Docker daemon', start/restart Docker Desktop.

Step 3 — Run a basic test container.

```
docker run --rm hello-world
```

Check: You should see the 'Hello from Docker!' message.

Action: If it fails, Docker itself is not ready. Fix Docker before attempting Kubernetes.

1.2 Check Kubernetes

Step 1 — Verify kubectl.

```
kubectl version --client
```

Step 2 — Check the active context.

```
kubectl config current-context
```

Check: For Docker Desktop it commonly shows docker-desktop.

Action: If the context is wrong, switch to the required cluster context.

Step 3 — Check cluster connectivity.

```
kubectl cluster-info
```

Check: Kubernetes control plane information should be returned.

Step 4 — Check the node.

```
kubectl get nodes
```

Check: The node should normally show STATUS = Ready.

Action: If NotReady, wait for Kubernetes to finish starting. If it remains NotReady, restart Kubernetes/Docker Desktop before continuing.

1.3 Check Helm

Verify Helm.

```
helm version
```

Check: Helm version information should appear.

Action: If the command is not recognized, Helm is not installed or is not available in PATH.

1.4 Check Existing Lab Resources

Check all pods.

```
kubectl get pods -A
```

Check services.

```
kubectl get svc -A
```

Check Helm releases.

```
helm list -A
```

Important: Old resources from an earlier attempt can cause errors such as 'name already in use', occupied ports, or duplicate Helm releases. Inspect before creating new resources.

2. Docker Troubleshooting

2.1 Docker Command Says Cannot Connect to Daemon

Step 1 — Confirm Docker Desktop is open.

Step 2 — Run:

```
docker info
```

Check: If Docker is healthy, server information is displayed.

Step 3 — If it still fails.

Action: Restart Docker Desktop. Wait until the engine is fully running, then retry `docker info`.

2.2 Docker Image Build Fails

Step 1 — Confirm you are in the correct folder.

```
dir    # Windows Command Prompt / PowerShell
ls     # Git Bash / macOS / Linux
```

Check: The Dockerfile and application files should be visible.

Step 2 — Confirm the filename is exactly Dockerfile. Avoid names such as Dockerfile.txt.

Step 3 — Build with a clear tag.

```
docker build -t student-app:v1 .
```

Step 4 — If build fails, read the first meaningful error. Typical causes include a wrong COPY path, missing requirements.txt, package installation failure, or invalid Dockerfile instruction.

Step 5 — Rebuild after correcting the problem.

```
docker build --no-cache -t student-app:v1 .
```

Step 6 — Verify the image.

```
docker images
```

Check: student-app with tag v1 should be listed.

2.3 Container Starts and Immediately Stops

Step 1 — Show stopped containers.

```
docker ps -a
```

Step 2 — Copy the container name or ID and inspect its logs.

```
docker logs <container-name-or-id>
```

Step 3 — Correct the application error shown in the logs. Examples: Python import error, missing package, wrong startup command, application crash.

Step 4 — Rebuild and rerun if application files changed.

```
docker build -t student-app:v1 .
```

2.4 Port Already Allocated / Address Already in Use

Step 1 — Check running containers.

```
docker ps
```

Step 2 — Find the container using the required host port.

Step 3 — Stop only that container if it is not required.

```
docker stop <container-id>
```

Step 4 — Or map to another host port.

```
docker run -p 5001:5000 student-app:v1
```

Check: The app is then accessed through host port 5001.

2.5 Container Is Running but Website Does Not Open

Step 1 — Confirm the container is running.

```
docker ps
```

Step 2 — Check the PORTS column. You should see a mapping such as 0.0.0.0:5000->5000/tcp.

Step 3 — Check logs.

```
docker logs <container-id>
```

Step 4 — For Flask, make sure the app listens on all interfaces.

```
app.run(host="0.0.0.0", port=5000)
```

Step 5 — Test again in the browser. Example: <http://localhost:5000>

3. Kubernetes Troubleshooting

For Kubernetes problems, do not guess. Use the following sequence:

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl get svc
```

3.1 kubectl Cannot Connect to the Cluster

Step 1 — Check context.

```
kubectl config current-context
```

Step 2 — Check cluster.

```
kubectl cluster-info
```

Step 3 — Check nodes.

```
kubectl get nodes
```

Step 4 — If connection is refused.

Action: Open Docker Desktop and confirm Kubernetes is enabled and running. Wait for the cluster to become ready before retrying.

3.2 YAML File Does Not Apply

Step 1 — Run the apply command and read the exact line/error.

```
kubectl apply -f deployment.yaml
```

Step 2 — Check indentation. YAML uses spaces. Keep child properties consistently indented; avoid tabs.

Step 3 — Check required fields. Common mistakes include wrong apiVersion, misspelled metadata/spec fields, and misplaced containers/ports.

Step 4 — Ask Kubernetes to validate without creating resources.

```
kubectl apply --dry-run=client -f deployment.yaml
```

Step 5 — Correct the YAML and apply again.

```
kubectl apply -f deployment.yaml
```

3.3 Pod Shows Pending

Step 1 — Identify the pod.

```
kubectl get pods
```

Step 2 — Describe it.

```
kubectl describe pod <pod-name>
```

Step 3 — Read the Events section at the bottom. Look for scheduling, resource, volume, or node errors.

Step 4 — Check node health.

```
kubectl get nodes
```

Step 5 — Fix the cause and recheck.

```
kubectl get pods
```

3.4 ImagePullBackOff / ErrImagePull

Step 1 — Describe the pod.

```
kubectl describe pod <pod-name>
```

Check: Read Events for the exact image name and pull error.

Step 2 — Verify the image exists locally if using Docker Desktop.

```
docker images
```

Step 3 — Verify image and tag in YAML.

```
image: student-app:v1
```

Step 4 — For a local lab image, use the intended pull policy.

```
imagePullPolicy: Never
```

Step 5 — Reapply.

```
kubectl apply -f deployment.yaml
```

Step 6 — Watch the pod.

```
kubectl get pods -w
```

Important: Use `imagePullPolicy: Never` only when the image is intentionally available to the local Kubernetes environment. Registry-hosted images should normally be pulled instead.

3.5 Pod Shows CrashLoopBackOff

Step 1 — Check pod.

```
kubectl get pods
```

Step 2 — Read application logs.

```
kubectl logs <pod-name>
```

Step 3 — If the container has already restarted, inspect previous logs.

```
kubectl logs <pod-name> --previous
```

Step 4 — Describe the pod.

```
kubectl describe pod <pod-name>
```

Step 5 — Fix the application/configuration error. Then rebuild the Docker image if necessary and restart/redeploy the workload.

Step 6 — Restart the deployment after a corrected image/configuration.

```
kubectl rollout restart deployment/<deployment-name>
```

Step 7 — Verify rollout.

```
kubectl rollout status deployment/<deployment-name>
```

3.6 Pod Running but Application Does Not Open

Step 1 — Confirm pods are Running.

```
kubectl get pods -o wide
```

Step 2 — Check the service.

```
kubectl get svc
```

Step 3 — Inspect service details.

```
kubectl describe svc <service-name>
```

Step 4 — Verify port relationships. For example: app listens on 5000 → targetPort 5000 → service port 5000 → NodePort 30080.

Step 5 — Verify service selector matches pod labels.

```
kubectl get pods --show-labels
```

Step 6 — Check service endpoints.

```
kubectl get endpoints <service-name>
```

Check: If endpoints are empty, the service selector is not matching the pods.

Step 7 — For the class NodePort example, test:

```
http://localhost:30080
```

3.7 Deployment Updated but Old Version Still Appears

Step 1 — Check deployment image.

```
kubectl describe deployment <deployment-name>
```

Step 2 — Check rollout.

```
kubectl rollout status deployment/<deployment-name>
```

Step 3 — Check pods and their age.

```
kubectl get pods
```

Step 4 — If using the same local tag after rebuilding, restart deployment.

```
kubectl rollout restart deployment/<deployment-name>
```

Step 5 — If the new version is faulty, rollback.

```
kubectl rollout undo deployment/<deployment-name>
```

3.8 Fast Kubernetes Status Guide

If you see	Meaning	Next action
Running	Container is running.	Check Service/network if app is inaccessible.
Pending	Pod has not been scheduled/started.	Describe pod and inspect Events.
ErrImagePull	Image could not be pulled.	Check image name, tag, registry/pull policy.
ImagePullBackOff	Repeated image pull failures.	Describe pod; fix image/pull issue.
CrashLoopBackOff	Application repeatedly crashes.	Check logs and previous logs.
Completed	Process finished successfully.	Normal for Jobs; unexpected for long-running web apps.

4. Helm Troubleshooting

4.1 Confirm Helm and Repositories

Step 1 — Verify Helm.

```
helm version
```

Step 2 — List repositories.

```
helm repo list
```

Step 3 — Update repository metadata.

```
helm repo update
```

4.2 Repository Not Found

Step 1 — Check whether the repository exists.

```
helm repo list
```

Step 2 — For the Prometheus community charts, add it if missing.

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
```

Step 3 — Update repositories.

```
helm repo update
```

Step 4 — Search for the chart.

```
helm search repo kube-prometheus-stack
```

4.3 Namespace Does Not Exist

Step 1 — Check namespaces.

```
kubectl get namespaces
```

Step 2 — Create the monitoring namespace if required.

```
kubectl create namespace monitoring
```

Step 3 — Retry the Helm installation.

4.4 Release Name Already Exists / Cannot Re-use a Name

Step 1 — List releases in all namespaces.

```
helm list -A
```

Step 2 — If the release already exists, inspect it.

```
helm status monitoring -n monitoring
```

Step 3 — If you intend to update the existing release, use upgrade rather than another install.

```
helm upgrade monitoring prometheus-community/kube-prometheus-stack -n monitoring
```

Step 4 — Only if you intentionally want to remove it, uninstall the specific release.

```
helm uninstall monitoring -n monitoring
```

4.5 Helm Install Failed

Step 1 — Check release status.

```
helm list -A
```

Step 2 — Check monitoring pods.

```
kubectl get pods -n monitoring
```

Step 3 — Describe the failed/pending pod.

```
kubectl describe pod <pod-name> -n monitoring
```

Step 4 — Read logs where applicable.

```
kubectl logs <pod-name> -n monitoring
```

Step 5 — Fix the underlying Kubernetes problem first. Helm is often only reporting a failure caused by a pod, volume, image, or resource issue.

5. Prometheus Troubleshooting

5.1 Check Monitoring Stack Health

Step 1 — Check Helm release.

```
helm list -n monitoring
```

Step 2 — Check pods.

```
kubectl get pods -n monitoring
```

Step 3 — If a pod is not Running, describe it.

```
kubectl describe pod <pod-name> -n monitoring
```

Step 4 — Check logs.

```
kubectl logs <pod-name> -n monitoring
```

5.2 Prometheus Query Returns 'No Data'

Step 1 — Do not immediately change Grafana. First confirm the metric exists in Prometheus.

Step 2 — Search a broad metric first.

```
kube_pod_status_phase
```

Step 3 — Inspect labels returned by the metric. Check the actual namespace, pod, container, job and phase values.

Step 4 — Add one filter at a time.

```
kube_pod_status_phase{namespace="default"}
```

Step 5 — Then add the pod regex.

```
kube_pod_status_phase{namespace="default",pod=~"student-app-deployment-.*"}
```

Step 6 — Then add the phase.

```
sum(kube_pod_status_phase{namespace="default",pod=~"student-app-deployment-.*",phase="Running"})
```

Important: In PromQL, `=` is an exact label match and `=~` is a regular-expression match. A pattern such as `student-app-deployment-.\*` therefore needs `=~`, not `=`.

5.3 CPU Query Returns No Data

Step 1 — Test the base metric.

```
container_cpu_usage_seconds_total
```

Step 2 — Add namespace only.

```
container_cpu_usage_seconds_total{namespace="default"}
```

Step 3 — Add the pod regex only after confirming the label exists.

```
container_cpu_usage_seconds_total{namespace="default",pod=~"student-app-deployment-.*"}
```

Step 4 — Apply rate and sum.

```
sum(rate(container_cpu_usage_seconds_total{namespace="default",pod=~"student-app-deployment-.*"}[1m]))
```

Step 5 — If extra filters cause No Data.

Action: Remove filters such as container/image one at a time and inspect the actual labels available in your environment.

5.4 Prometheus Target Is DOWN

Step 1 — Open Prometheus Targets page and identify the DOWN target.

Step 2 — Check the corresponding Kubernetes pod/service.

```
kubectl get pods -n monitoring
kubectl get svc -n monitoring
```

Step 3 — Describe the affected pod.

```
kubectl describe pod <pod-name> -n monitoring
```

Step 4 — Check logs.

```
kubectl logs <pod-name> -n monitoring
```

Step 5 — Resolve the pod/service problem and verify that the target becomes UP.

6. Grafana Troubleshooting

6.1 Grafana Does Not Open

Step 1 — Find the Grafana pod.

```
kubectl get pods -n monitoring
```

Step 2 — Confirm it is Running.

Step 3 — Check Grafana service.

```
kubectl get svc -n monitoring
```

Step 4 — If using port-forwarding, start it in a terminal and keep that terminal open.

```
kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80
```

Step 5 — Open Grafana.

```
http://localhost:3000
```

Step 6 — If the browser cannot connect, check whether the port-forward command is still running and whether port 3000 is already in use.

6.2 Retrieve Grafana Admin Password

Step 1 — List secrets to confirm the Grafana secret name.

```
kubectl get secrets -n monitoring
```

Step 2 — For a standard kube-prometheus-stack release named monitoring, retrieve the password.

```
kubectl get secret monitoring-grafana -n monitoring -o jsonpath="{.data.admin-password}"
```

Step 3 — Decode the Base64 value. On PowerShell you can decode using PowerShell; on Git Bash/Linux/macOS pipe to base64 --decode. The exact command depends on the terminal.

Important: Do not hard-code or share a real production password. This procedure is for the local classroom/lab stack.

6.3 Grafana Shows No Data

Step 1 — Copy the panel's PromQL query.

Step 2 — Run the same query directly in Prometheus.

Step 3 — If Prometheus also shows No Data.

Action: The problem is the metric/query/labels. Fix Prometheus query first.

Step 4 — If Prometheus returns data but Grafana does not. Check the selected Grafana data source, dashboard time range, panel query, variables and refresh interval.

Step 5 — Set a reasonable recent time range. For a live practical, try Last 5 minutes or Last 15 minutes.

Step 6 — Refresh the dashboard.

6.4 Prometheus Data Source Fails

Step 1 — Confirm Prometheus is running.

```
kubectl get pods -n monitoring
```

Step 2 — Confirm Prometheus service exists.

```
kubectl get svc -n monitoring
```

Step 3 — In Grafana, verify that the selected Prometheus data source points to the monitoring stack's Prometheus service.

Step 4 — Use Grafana's Save & Test.

Check: The data source should report a successful connection.

7. Emergency Recovery Flow During Practical

If you are completely stuck, follow these steps in exactly this order. Stop as soon as you find the failing layer.

Step	Check	Command / Test	Decision
1	Docker engine	docker info	If this fails, fix/restart Docker Desktop.
2	Kubernetes node	kubectl get nodes	Node must be Ready.
3	Application pods	kubectl get pods	Identify Pending / ImagePullBackOff / CrashLoopBackOff.
4	Pod details	kubectl describe pod <pod-name>	Read Events at the bottom.

5	Application logs	kubectl logs <pod-name>	Fix the actual application/config error.
6	Service	kubectl get svc	Check type, ports and NodePort.
7	Service endpoints	kubectl get endpoints <service-name>	Empty endpoints usually mean selector/label mismatch.
8	Helm	helm list -A	Confirm monitoring release exists.
9	Monitoring pods	kubectl get pods -n monitoring	Prometheus/Grafana pods should be healthy.
10	Prometheus	Test the metric broadly first	Add label filters one at a time.
11	Grafana	Run same PromQL in Prometheus	Only debug Grafana after Prometheus returns data.

8. Safe Diagnostic Commands — Quick Reference

Docker

```
docker version
docker info
docker images
docker ps
docker ps -a
docker logs <container>
```

Kubernetes

```
kubectl get nodes
kubectl get pods
kubectl get pods -A
kubectl describe pod <pod>
kubectl logs <pod>
kubectl get svc
kubectl get endpoints <service>
kubectl get deployments
```

Helm

```
helm version
helm repo list
helm repo update
helm list -A
helm status <release> -n <namespace>
```

Monitoring

```
kubectl get pods -n monitoring
kubectl get svc -n monitoring
kubectl describe pod <pod> -n monitoring
kubectl logs <pod> -n monitoring
```

9. Commands Students Should NOT Run Randomly

These actions can destroy the working lab state. Use them only when instructed by faculty/lab staff:

- Docker system-wide prune/cleanup commands that remove images, containers or volumes.
- Reset Kubernetes Cluster / factory reset in Docker Desktop.
- Deleting all namespaces, all deployments, or all pods without understanding the impact.
- Uninstalling the monitoring Helm release just because one pod is failing.
- Changing many YAML fields simultaneously before identifying the original error.

10. Final 2-Minute Practical Checklist

- ☐ Docker Desktop is running.
- ☐ `docker info` works.
- ☐ `kubectl get nodes` shows Ready.
- ☐ Required Docker image/tag exists.
- ☐ Application pods are Running.
- ☐ Service exists and ports are correct.
- ☐ Application URL/NodePort works.
- ☐ Helm monitoring release exists.
- ☐ Monitoring pods are healthy.
- ☐ Prometheus query returns data.
- ☐ Grafana dashboard displays the expected metric.